

Project 3 - Duck Park

Oliver Boorstein
University of Oregon

June 5, 2026

1 Introduction

This project implements Duck Park, a small amusement park simulation in C using Pthreads. Passenger threads explore the park, wait for tickets, enter the ride queue, board cars, ride, and unboard. Car threads repeatedly load, run, and unload. The final part also adds a separate monitor process that prints queue state using IPC. Compared to Project 2, this project was less about scheduling and more about synchronization.

2 Background

The main idea behind the project is that the park is a shared model being changed by many threads at the same time. A passenger cannot board whenever it wants, and a car cannot leave whenever it wants. Each action depends on the current queue state, the car state, the passenger state, and whether the park is still open. Because of that, I needed more structure than just starting a group of threads and locking around a few variables.

The most useful design decision I made was representing the park logic through a shared `data_model` and functions. The `Park` struct stores the configuration values, shared queues, condition variables, passengers, cars, and the `park_open` flag. The `Passenger` and `Car` structs each have explicit state fields. This made the synchronization easier to reason about because most thread functions operate on one shared model instead of passing many separate globals around.

3 Implementation

I split the project into the same three parts as the assignment. Part 1 verifies the basic simulation with one passenger and one car. Part 2 expands the same structure to multiple passengers and cars. Part 3 keeps that implementation and adds monitoring through a child process. Most of the interesting code is in the final version because the earlier parts are simpler versions of the same design.

I implemented the passenger lifecycle as a small state machine. A passenger starts by exploring, then enters the ticket queue, waits to board, rides, waits to unboard, and finishes the cycle. I also treated the car lifecycle as a state machine: waiting to load, loading, running, waiting to unload, and unloading. This was important because the synchronization rules are mostly rules about transitions. For example, a passenger can only move into `PASSENGER_CAN_BOARD` when a loading car removes that passenger from the ride queue. A passenger can only move into `PASSENGER_CAN_UNBOARD` when the assigned car invokes `unload()`.

The queues are also part of the synchronization design. The ticket queue controls who may receive the next ticket, while the ride queue controls who may board next. A ticket is only issued when the passenger is at the front of the ticket queue and the ride queue is not full. This matches the project requirement that passengers cannot leave a queue early and that ticket issuing must stop when the ride queue reaches its maximum size. The queue implementation is a circular array, but all queue operations happen while holding the park mutex, so pushing, popping, peeking, and removing remain atomic from the threads' perspective.

I used two extra queues for car loading and unloading. The load queue makes sure only one

car invokes the loading section at a time. Each car pushes itself into the load queue and waits until it is at the front. After that, it boards passengers from the ride queue until the car is full, the park closes, or the partial-car timeout expires. The unload queue solves the rule that cars cannot pass each other. Multiple cars can be running at the same time, but each car has to wait until it reaches the front of the unload queue before it can notify its passengers to unboard.

Condition variables do most of the blocking. There are park-level condition variables for ticket, ride, load, and unload events, plus a per-passenger condition variable. The broad condition variables are useful when a shared queue changes and many threads might need to re-check their loop condition. For example, when a passenger enters the ride queue, waiting cars are woken through the ride condition variable. When a passenger boards and creates room in the ride queue, ticket waiters are woken through the ticket condition variable. The per-passenger condition variable is more precise. Cars signal the exact passengers who may board or unboard, instead of waking every passenger for every car event.

The most important rule I followed with these condition variables is that every wait is inside a loop. This handles spurious wakeups, but it also handles normal wakeups where the event was useful for another thread and not this one. For example, many cars may wake up when the load queue changes, but only the car at the front can continue. Similarly, many passengers may be waiting for tickets, but only the front passenger can get a ticket when the ride queue has space.

4 Shutdown and Monitoring

Closing the park was one of the trickier parts because some threads may be asleep while exploring, some may be waiting for tickets, some may be waiting to board, and cars may be waiting on either loading or unloading. I handled this by having the main thread sleep for the configured park duration, then set `park_open` to false while holding the lock. After that it broadcasts to all park condition variables and all passenger condition variables. Passenger threads check the flag at each blocking point and leave their queues if needed. Car threads stop starting new rides once loading fails, but a car that already departed still finishes its run and unloads its passengers. This prevents passengers from being stranded in the riding state during shutdown.

Part 3 adds IPC monitoring with a child process and a pipe. The parent process owns the real park state and periodically creates a snapshot while holding the park mutex. The snapshot includes the current time, ticket queue contents and capacity, ride queue contents and capacity, and whether the park has closed. That formatted text is written to the pipe. The monitor process reads from the other end and writes the snapshots to standard output. I chose this because it keeps the monitor simple and avoids sharing raw thread-owned memory across processes. The monitor only displays state; it never changes the simulation.

5 Performance

Performance is good for the assignment scale. The simulation is not CPU-heavy because passenger and car threads mostly sleep or block on condition variables. There is no busy waiting in the main synchronization paths. The main cost is that I protect all park state with one mutex, so only one thread can change queues or states at a time. For this project that trade-off is worth it because

the critical sections are short and the single lock makes correctness much easier. Splitting the lock across several queues and cars might improve concurrency, but it would also make deadlock ordering much harder.

The monitor adds very little overhead. It takes a snapshot once per second and writes a small formatted string through a pipe. The parent holds the mutex only while reading queue state into the buffer, then releases it before continuing the simulation. This makes the monitor useful for debugging without changing the behavior very much.

6 Conclusion

Overall, this was the hardest project so far. Project 1 was mostly parsing and syscalls, and Project 2 was mostly signal ordering. In this project, I had to model the actual park rules carefully enough that the synchronization followed from the data. The final version uses explicit passenger and car states, queue ordering, condition variables, broadcasts, and a pipe-based monitor process. The result supports multiple passengers and cars, partial-car departures after the wait period, ordered unloading, clean shutdown, and live queue monitoring.